HOTEE: Building Secure and Logarithmic-complexity Search Indexes for Hybrid-trust Encrypted Databases

Abstract

Place holder. Driven by the rapid advancement of cloud computing, organizations are increasingly moving their sensitive data to cloud services (e.g., databases), which offers elasticity, high availability, and low cost. For example, the US state and local governments use AWS to host databases for criminal records [1]; the Harvard global health institute uses Google cloud to store healthcare data (e.g., hospitalizations, ventilator utilization) in databases for COVID-19 forecast [3]. Besides, to generate sufficient data insights and give precise analytics, organizations have grown increasingly collaborative to share their data and make joint queries on all databases [41, 49].

1 Introduction

Driven by the rapid advancement of cloud computing, organizations are increasingly moving their sensitive data to cloud services (e.g., databases), which offer elasticity, high availability, and low cost. For example, the US state and local governments use AWS to host databases for criminal records [1]; the Harvard global health institute uses Google cloud to store healthcare data (e.g., hospitalizations, ventilator utilization) in databases for COVID-19 forecast [3]. Besides, to generate sufficient data insights and give precise analytics, organizations have grown increasingly collaborative to share their data and make joint queries on all databases [41, 49].

Owing to the untrusted nature of the cloud [46], organizations typically require their uploaded data in cloud databases to keep confidential from adversaries, including both the insiders with administrative privileges and attackers who compromise the cloud database services [2, 43]. Existing sophisticated databases (e.g., MySQL) employ access control and data-at-rest encryption [35], which can be easily bypassed by adversaries and exfiltrate plaintexts during query execution.

To protect data confidentiality on cloud databases, both the industry and academia have expedited the developments of *encrypted databases* [2, 19, 24, 33, 37, 39, 43, 47], and existing work can be divided in two categories. The first category (e.g., EnclaveDB [39], Azure EdgelessDB [2], Enclage [43]) uses Trusted Execution Environment (TEE), in particular Intel SGX [16], to execute queries on plaintexts in a hardware-level isolated region called enclave [16]. *TEE databases* can operate on plaintexts with high efficiency, but sustain the risk of trusting hardware vendors and long-lasting debates on side-channel attacks [26] that deduce plaintexts at runtime.

The second line of work (e.g., CryptDB [37], HElibDB [4], FHEDB [33]) relies on homomorphic encryption (HE), especially fully HE schemes (e.g., BGV [21]), to allow databases compute arbitrary aggregations over homomorphic data, and only the users can decrypt to see plaintext results. With a cryptographic root-of-trust, *HE databases* provide stronger confidentiality guarantees than TEE databases, hence attracting an increasing number of organizations. For instance, Canada government has planned to link confidential data dispersed across the government departments and public institutions using HE databases [50].

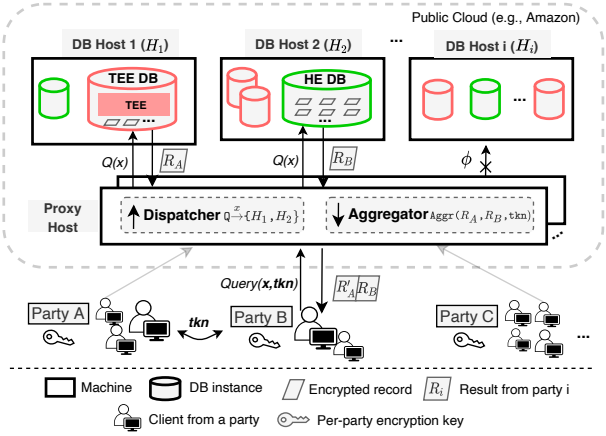


Figure 1: HOTEE's hybrid-trust multi-party joint query scenario.

Since different organizations can have different trusts and preferences in encrypted databases, motivated by the growing demand for multi-party collaborative analytics [47, 49], we consider a hybrid-trust multi-party joint query scenario wherein a group of authenticated parties shares sensitive data stored in hybrid encrypted databases (i.e., either a TEE or HE database) on the cloud and all databases jointly answer queries from one of the parties (Figure 1). Specifically, queries are first filtered and forwarded to relevant databases in which the queried content exists, then databases execute searches on encrypted data to produce partial results, and finally all partial results are securely aggregated and returned to the querier. Such a computing model can support a wide range of secure applications. For instance, physicians can make distributed queries to multiple distinct medical institutions owning sensitive symptom records for collaborative diagnosis.

Unfortunately, to the best of our knowledge, despite much academic and industrial effort in developing systems for distributed joint queries on encrypted databases [9, 47, 49], none of them are designed for the hybrid-trust scenario where TEE and HE databases coexist. This is due to two major challenges that stem from the utility gap between TEE and HE databases.

First, as TEE databases directly operate on plaintexts in enclaves, so they support all typical query operations [31], including *equality*, *range*, *aggregation*, or their combinations, by searching on well-studied *logarithmic-complexity* indexes (typically, a B^+-tree) in enclaves. However, none of existing HE databases can efficiently support all these operations, bringing a cascading effect of limiting TEE databases' utility in the hybrid-trust joint query scenario.

Specifically, existing HE databases use two methods to support operations other than HE-enabled aggregation locally. One line of work (e.g., HElibDB [4], FHEDB [33]) uses specific cryptographic functions (e.g., FLT [4]) to evaluate the equality of homomorphic data by scanning the whole dataset with *linear-complexity*, which is inefficient given massive (and constantly growing) amounts of outsourced data and does not support range search; another line of work (e.g., CryptDB [37]) uses distinct encryption methods to support either operations, i.e., use deterministic encryption [10] for equality search and order-preserving encryption [5] for range search with logarithmic-complexity , but only supports static data with known distribution (i.e., write-limited) and cannot support both operations simultaneously.

The second challenge is to enable the proxy to securely exclude non-matching data in irrelevant distributed encrypted databases. Specifically, in traditional plaintext distributed databases (e.g., BigTable [14], PostgreSQL [38]), queries are first committed to an index server (i.e., proxy) equipped with a membership index, which analyzes query conditions to early-exclude non-matching data in irrelevant databases before query execution to reduce unnecessary I/O operations and accelerate distributed query processing.

Nevertheless, a new confidentiality-locality gap arises when querying distributed encrypted databases: organizations' sensitive data must be kept confidential from a third-party proxy, and the proxy should locate encrypted databases that comprise matching data. Although existing systems [11, 19] propose to use Bloom filter [12], a space-efficient index to check the membership of queried data in encrypted databases, however, Bloom filters only support equality checks, and cannot handle multi-party data membership checking in the hybrid-trust joint query scenario.

In this paper, we propose HoTEE[1], the first hybrid-trust joint query framework that efficiently performs all typical query operations (equality, range, aggregation) in heterogeneous encrypted databases while ensuring end-to-end data confidentiality. To tackle the aforementioned challenges, we present a new *stealth index* abstraction called *MeRF-Tree*, which supports logarithmic-complexity indexing for equality and range searches on encrypted data simultaneously and enables non-matching data exclusion for general queries, solving two challenges with one abstraction in a unified manner.

To build MeRF-Tree stealth index, our key insight is that filters have an intrinsic stealth feature of hiding plaintexts, e.g., Bloom filter maps plaintexts to random positions in distinct bitmaps as `"1"`, checks membership by verifying whether every position for queried data is `"1"`. Inspired by filter's bitwise stealth feature, we can stack filters into a Merkle-tree style data structure where each node's filter is the *OR* gate computation result of its child nodes' filters, such that the resultant tree can support logarithmic-complexity indexing for equality search without revealing plaintexts. Furthermore, to support stealth range search, we propose *stealth bias* to patch a new range property to index nodes' filters, such that filters can preserve the same numerical orders as their corresponding plaintexts. Consequently, we term the stealth index as Merkle Range Filter Tree (i.e., MeRF-Tree).

Unfortunately, even with the stealth index, another security problem arises when aggregating multi-party data at proxy, given that the retrieved data may stem from different organizations and is encrypted under distinct keys which cannot be directly shared. A trivial approach is to set up TEE enclaves at the proxy, transform all retrieved data by decrypting and then re-encrypting with the querier's key pre-stored in proxy enclaves. However, owing to different trusts of organizations, the retrieved homomorphic data from HE databases is forbidden to decrypt as plaintexts in enclaves (§2.2), hence cannot be transformed to querier-decryptable data in proxy enclaves.

To remedy this issue, we identified a cryptographic tool, *key switch* [21] (an algorithm for reducing homomorphic data noises), which allows us to securely switch encryption keys of homomorphic data without revealing plaintexts. Based on the tool, HoTEE designs a secure distributed key switch protocol which securely transforms multi-party data for aggregation without breaking the hybrid trusts, and protects data confidentiality even when proxy and querier collude (§4).

We implemented HoTEE on CryptDB codebase [37], a mature modular framework for evaluating distributed encrypted databases. We built two HE databases with HoTEE by augmenting prominent databases: HoTEE-MySQL and HoTEE-WiredTiger. MySQL [34] is the most widely used SQL backend for encrypted databases; WiredTiger [15] is a popular key-value store that is the default NoSQL backend for MongoDB. We run HE database with an open-source TEE database Azure EdgelessDB [2] for hybrid-trust joint queries. We compared HoTEE with two famous HE databases CryptDB [37] and HElibDB [4] using typical workloads (e.g., TPC-C [29]) evaluated in relevant systems [6, 37, 48]. Evaluation shows that:

- **Utility**. HoTEE is the first generic framework that enables distributed queries on multi-party encrypted databases, supports general queries with logarithmic search complexity and preserves end-to-end confidentiality of all parties' data in a hybrid-trust computing scenario (§6).
- **Efficiency**. HoTEE-MySQL achieved xx% and xx% lower latency on average for equality search and range search than baselines; HoTEE-WiredTiger scaled out to larger datasets and more parties with almost linearly increased throughput

[1] HoTEE stands for Homomorphic-TEE hybrid-trust joint query.

	HoTEE	MySQL [34]	EnclaveDB [39]	EdgelessDB [2]	Cryptdb [37]	HElibDB [24]	FHEDB [33]	HybrTC [47]	Dory [19]
Data & query confidentiality	✓	✗	✓	✓	✓	✓	✓	✓	✓
Hybrid-trust joint query	✓	✗	✗	✗	✗	✗	✗	✓°	✓°
Equality search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	n/a.	$O(n)$
Range search	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	n/a.	n/a.	n/a.	n/a.
Insertion	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$	$O(1)$	n/a.	$O(1)$

°: Potentially support, but not explicitly handle hybrid-trust joint queries.

Table 1: Comparison of HoTEE and existing systems. "**n/a**." means not applicable as they do not support such query operations.

while querying latency remained stable (§6).

• **Extensibility**. HoTEE supports various types of databases, including both relational and NoSQL databases (e.g., MySQL and WiredTiger).

Our main contribution is MeRF-Tree, a new stealth index abstraction that supports logarithmic-complexity indexing for general search queries (e.g., equality and range) on distributed encrypted databases without revealing plaintexts. This enables HoTEE to securely bridge the utility gap between heterogeneous encrypted databases, and allows early exclusion of non-matching data under various search conditions. Our other contributions include a secure distributed key switch protocol, which is the first effective attempt to tackle hybrid-trust data aggregation, making HoTEE unique to facilitate multiple parties with distinct trusts on encrypted databases to securely share data and make joint queries in various mission-critical applications (e.g., medical diagnosis [28, 30], governmental data collection [9, 50]). HoTEE's source code is released on `github.com/osdipxxx/HoTEE`.

In the rest of this paper: §2 introduces background; §3 gives an overview of HoTEE; §4 describes HoTEE's design; §5 covers HoTEE's implementation; §6 shows our evaluation; §7 discusses related work and §8 concludes.

2 Background and Motivation

2.1 Securing Data for Cloud Databases

Today, organizations are increasingly moving their constantly growing amounts of data to cloud databases [1, 3]. Yet, many organizations still decline to widely adopt cloud databases for data management due to severe confidentiality concerns. To address the security issue, techniques have emerged that allow queries to be run against encrypted data kept in cloud databases. Two prominent approaches consist of hardware-based solutions using *Trusted Execution Environment* (TEE) like Intel SGX [16] and software-based solutions using *Homomorphic Encryption* (HE) like BGV [21].

Intel SGX. SGX [16] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called *enclave*, where data and code execution cannot be seen or tampered with from outside. SGX has been widely adopted in recent encrypted databases [2, 39, 43] because (1) SGX provides sophisticated API including sealing and attestation, and (2) the latest SGX 2.0 greatly expands enclave memory capacity (up to 512GB/CPU [25]) to fit memory-intensive applications including databases [43].

Homomorphic BGV. To avoid trusting one TEE hardware vendor, some organizations are seeking a cryptographic root-of-trust from homomorphic encryption (HE), and are willing to pay for the deficiencies of using HE-based solutions, especially fully HE schemes like BGV [21]. BGV allows arbitrary aggregation (e.g., `sum`, `avg`) on encrypted data (with noises) without requiring a secret key for decryption, and the computation result can be fully revealed by the owner of the secret key [21]. BGV uses *key switch* algorithm to reduce ciphertext's noises, which is leveraged by HoTEE to aggregate multi-party data encrypted by distinct secret keys (§4.3).

2.2 Heterogeneous Encrypted Databases

To make a clear comparison, we summarize the representative systems and compile their features in Table 1. Note that, the traditional MySQL database uses access control and data-at-rest encryption to protect data [35], failing to provide data and query confidentiality (i.e., process encrypted queries on the encrypted data to defend against adversaries described in §1), but enjoys full-fledged search functionalities with logarithmic-complexity indexing on plaintexts using B^+-tree [34].

TEE databases. The first category is TEE-based encrypted databases. EnclaveDB [39] makes user data completely inaccessible by implementing in-memory storage and query engine in enclaves, protecting data and query confidentiality and supporting logarithmic-complexity operations using a TEE-shielded B^+-tree. Azure EdgelessDB [2], and Alibaba Enclage [43] consider on-disk storage (sealed by SGX) and use B^+-tree for efficiently indexing as well: Enclage shields most B^+-tree nodes in plaintext within enclaves; EdgelessDB stores an AES-encrypted B^+-tree outside enclaves, and performs searches by on-demand pulling index nodes to enclaves and then decrypting and comparing each node for indexing.

HE databases. Different from TEE databases that decrypt and then search plaintexts on a fully (or partially) shielded B^+-tree in enclaves, the homomorphic data is forbidden to decrypt and search in enclaves because HE database users demand a cryptographic root-of-trust thus distrust TEEs (§2.1).

To build search indexes for HE databases, CryptDB [37] uses deterministic encryption [10] for equality search by linearly scanning all data and uses order-preserving encryption [5] (OPE) for range search. Since OPE can preserve the original orders of plaintexts, one can use off-the-shelf indexes like B^+-tree for OPE ciphertexts same as plaintexts. However, OPE does not support equality search, and CryptDB's solution (i.e., onion encryption) becomes inefficient when the workload is mixed [45]. Besides, OPE assumes the plaintext domain distribution is fixed [32], which contradicts the dynamic nature of database writes. HElibDB [4] and FHEDB [33] use specific cryptographic tools to directly perform equality searches on homomorphic data with linear-complexity but lack support for range search on homomorphic data.

Hybrid-trust joint query. The aforementioned systems are designed for a single machine and do not support distributed queries on multiple hybrid-trust databases. HybrTC [47] integrates TEE with MPC [17] to support multi-party data analytics wherein users only trust their own TEE databases, but supports neither equality nor search queries on HE databases. Dory [19] supports only equality search for encrypted keywords by performing linear scans on bloom filters [12] because the bloom filter has limited expressiveness which tells only the existance of searched data on an encrypted database.

2.3 Motivation and Opportunities

From the survey results reported above, we summarize that different organizations can have different trusts and preferences in encrypted databases. Thus, in this paper, we envision the first hybrid-trust scenario where users make distributed queries to heterogeneous encrypted databases. Such a computing model brings two unique challenges. First, HoTEE should support general query operations with logarithmic search complexity on HE databases for joint query. Second, HoTEE should support encrypted query filtering and multi-party data aggregation same as distributed plaintext databases.

To tackle the challenges, we summarize what we considered opportunities to make the hybrid-trust joint query problem easier. First, we observe that filters used by traditional databases have an intrinsic stealth feature of hiding plaintexts while providing membership information, which can serve as a basic block to build an encrypted search index. Second, we can employ traditional homomorphic key switch algorithm to securely aggregate multi-party data, even if the algorithm has an orthogonal goal of reducing homomorphic data noises.

3 Overview

3.1 System Model

Entities. As shown in Figure 1, HoTEE has three roles: *host*, *proxy*, and *client*. Multiple clients (e.g., physicians) form a single party (e.g., hospital), and multiple parties outsource their encrypted data to database instances (for short, database) run on public cloud hosts for encrypted query service. Same as plaintext distributed databases [14, 38], clients first submit queries to a proxy, which profiles encrypted queries, filters query conditions, and then re-directs queries to relevant databases to retrieve encrypted data. The retrieved data is aggregated at the proxy and then returned to clients.

Data model. Without losing generality, we use the relational data model as an example to illustrate our design in the following discussions. We will discuss the extensibility in §7.1.

Specifically, a table T contains n columns (or attributes) $CL = \{c_0, c_1, ..., c_{n-1}\}$ and has a primary key $PK \in CL$. Same as [39,43], we consider the values in all columns are confidential, which means that each value should be encrypted before being sent to a database. For a record (i.e., a row) r, its value v_i in column c_i is ciphertext $E(v_i)$, where E represents the encryption scheme employed by a party: for TEE databases, E is typically AES-GCM [27] and the party's encryption key will be provisioned to SGX enclaves when computation over plaintext is needed; for HE databases, E is typically BGV [21], and the party never reveals its BGV encryption key to databases or other parties participated in joint queries.

Note that, same as existing encrypted databases [2, 39, 43], columns can be encrypted by distinct encryption keys to further strengthen confidentiality; it is also feasible to have plaintext columns if cross-column correlations are omitted [7, 8].

Storage model. Same as existing work [19, 43], in HoTEE, we focus only on the search system for end-to-end encrypted databases and not on the design of underlying storage engine. Many optimized storage engines [23, 42, 43] already exist and are used in encrypted databases. HoTEE is built on top of these storage engines, so below we abstract out functions that should be supported by an underlying storage engine.

Recall the aforementioned data model, we assume a table T with n columns $CL = \{c_0, c_1, ..., c_{n-1}\}$ and is indexed on the primary key c_0 (i.e., $PK = c_0$). A record r can be depicted as $[E(k), E(v_1), ..., E(v_{n-1})]$ where k is the value for c_0. The storage engine should provide the following API:

- EqualGet(E(k)): Retrieve the row r from T given a key k.
- RangeGet(E(k_s),E(k_e)): Retrieve all rows $\{r\}$ from T whose keys are between E(k_s) and E(k_e).
- FullGet(): Retrieve all rows $\{r\}$ from T.
- Put(r): Insert (or update) a row r to T if its key k is non-existent (or existent).
- Delete(E(k)): Delete the row r from T given a key k.

Note that, same as [2,37,43], HoTEE also supports searching on non-primary keys by building secondary indexes [40].

3.2 The HoTEE API

HoTEE provides a library called LibHoTEE, which contains basic API (Table 2) that facilitates heterogeneous encrypted databases to be deployed in HoTEE to jointly answer general queries. We will discuss how to integrate an encrypted database into HoTEE framework using LibHoTEE in §4.2. The data processing API consists of functions for clients to submit general queries and for proxy and databases to process

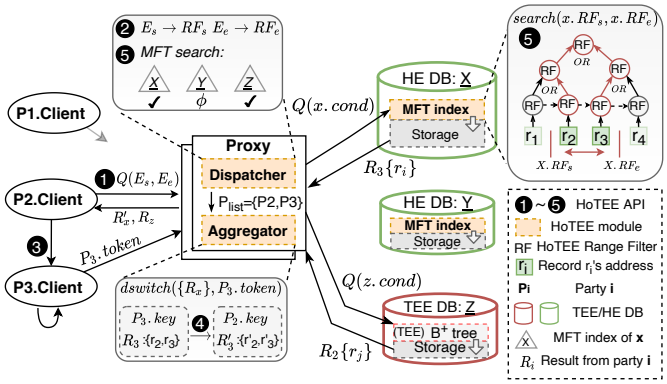


Figure 2: HoTEE's architecture, details described in §3.4.

encrypted queries (§4.3). The indexing API enables the proxy and databases to search encrypted data with HoTEE's stealth index abstraction: if called by the proxy, `MFTIdx(k)` returns *True* if the key k exists in a database; if called by the host, `MFTIdx(k)` returns the address of k in storage and calls the storage API (§3.1) to retrieve local data (§4.2).

3.3 Threat Model

HoTEE focuses on protecting the end-to-end confidentiality of all parties' data stored at database hosts. All roles in HoTEE are considered to be semi-honest. Specifically, same as existing single-party encrypted databases [4, 33, 37], we consider (1) a semi-honest database host which follows HoTEE's protocols but passively observes stored data, index items, and query results to deduce plaintexts, and (2) a semi-honest proxy which is curious about clients' queried data and query results. Similar to existing multi-party databases [19, 47], we consider all parties to be semi-honest, which means that (1) data outsourced to hosts is correct, and queries from parties' clients are authenticated through out-of-band services like [46], and (2) even if queries are authenticated, a party might still be curious about all sensitive data stored at hosts, and may collude with the proxy to deduce plaintext data from other parties.

Same as [2, 6, 37, 43], we do not protect metadata security. Specifically, HoTEE does not hide numerical orders between two ciphertexts, since existing oblivious approaches can lead to impractical performance penalty [18, 20]. Also, protecting coarse statistical properties such as the names of columns, the volume of indexes, or the length of values, is out of our scope. Same as [19, 37], we assume the underlying storage engine (§3.1) is maliciously secure, which can always retrieve the correct data requested by hosts. Providing such a guarantee (e.g., detecting rollback and fork attacks) has being well-studied in many storage systems [13, 44].

3.4 HoTEE's Workflow Overview

Figure 2 shows HoTEE's workflow with three phases.

• **Phase 1: query dispatch.** Typically, distributed databases use a proxy to dispatch client's queries to relevant database instances (for short, databases) for cost-efficiency. In HoTEE, clients submit signed queries (❶) to the proxy via a TLS-enabled link. The proxy will drop the connection if the client sends malformed queries (e.g., with invalid signatures from unknown parties) to avoid the client mouting DoS attacks on the proxy. For performance, queries are encrypted via AES, which are implemented efficiently in hardware.

LibHoTEE API	Role	Description
Data Processing API		
❶ `newQry(COND)`	CL[1]	Submit a new query with encrypted conditions (e.g., $E(k)$) to HoTEE's proxy (§4.2).
❷ `transform(E(K))`	PR[2]	Transform a key k to HoTEE's searchable index key (§4.2).
❸ `reqTok(KS)`	CL	Request new tokens from other clients for key switch (§4.3).
❹ `dSwitch(TK,D)`	PR	Run distributed key switch protocol to aggregate multi-party encrypted data (§4.3).
Indexing API		
❺ `MFTOp(COND)`	CL, DB[3]	Read/write by searching on HoTEE's MFT index (§4.2).

[1] **CL**: CLient [2] **PR**: PRoxy [3] **DB**: DataBase instance

Table 2: The API provided by LibHoTEE.

We let HoTEE's proxy run a dispatching thread to determine which encrypted databases should be forwarded to. In Figure 2, we use an example of range query Q to show the workflow. The proxy decrypts range conditions E_s and E_e, transforms them into HoTEE's confidential index primitive *range filter*, denoted as RF_s and RF_e (❷). These two range filters serve as the identifiers of ciphertexts, which are *deterministic* and *order-preserving*, hence can be leveraged for equlity search and range search without revealing plaintexts (§4.2). HoTEE searches RF_s and RF_e in pre-stored *MFT* indexes (introduced in **Phase 2**) for each database (❻), forwards Q to a database d if the searching result on d's MFT index is not ϕ, and generates a list of parties P_{list} that Q requested, to the aggregating thread (introduced in **Phase 3**).

• **Phase 2: local search.** Queries are forwarded to TEE or HE databases for local encrypted data searching, and both encrypted databases follow the separation of storage and unclustered indexes same as [43], so that HoTEE can inherit the mature safety and performance optimizations from existing storage systems [44]. Upon receiving Q, a TEE database decrypts query conditions in enclave, searches plaintext conditions in a TEE-shielding B^+-tree (red dotted box) where each tree node's value is plaintext, and calls the storage API (§3.1) to retrieve data. The on-plaintext B^+-tree searching brings TEE databases a natural advantage of supporting a full set of query operations, including *equality*, *range*, and *aggregation*.

For HE databases, different from existing systems that use specific cryptographic functions to support designated query operations with high-complexity (§2.2), HoTEE searches RF_s and RF_e on *merkle range filter tree* (MFT), a new confidential index that bridges the generality gap with TEE databases (❻). MFT is build on top of HoTEE's index primitive *range filter* (RF), by stacking RFs into a merkle-tree style structure where

each tree node's value is the *OR* gate computation result of its child nodes. In MFT, leaf nodes (nodes in the deepest tree layer) point to storage addresses of corresponding records, and all leaf nodes are linked for fast data retrieval same as traditional B$^+$-tree (§4.2).

• **Phase 3: data aggregate. Phase 3** runs in parallel with **Phase 2**. In **Phase 3**, an aggregating thread running at the proxy executes HoTEE's distributed key switch protocol to securely convert the retrieved encrypted data from multiple parties into client-decryptable data for usability (§4.3). In the example of Figure 2, the databases' searched results contain R_2 from party P_2, and R_3 from party P_3. Since Q is submitted by P_2's client, the aggregating thread fetches a secure token from P_3 (❸∼❹); upon receiving R_3, to ensure data condidentiality, our protocol securely converts R_3 into P_2's encryption key $P_2.key$ using that token, without having to decrypt R_3 (❺). By doing so, clients from different parties can securely decrypt retrieved data from other parties without directly sharing their sensitive encryption keys.

Overall, the highlight of HoTEE's workflow stems from achieving generality and low complexity on hybrid encrypted databases in semi-honest environments, and we illustrate in two aspects. Firstly, HoTEE's new MFT index tackles a full set of query operations (equality, range, and aggregation) on encrypted data, and MFT achieves $O(\log n)$ time complexity (n is the number of indexed keys) on equality and range search. Since aggregation is not a search operation on indexes, its complexity depends on the used algorithm (e.g., BGV [21] in HE databases). Besides, MFT can be generalized to support NoSQL databases and general types of attribute (e.g., real-values and string, see §7.1). Secondly, different from existing single-party encrypted databases [37, 43], HoTEE is the first framework to securely aggregate multi-party encrypted data without revealing decryption keys in a semi-honest environment (§4.3), facilitating multiple parties to share sensitive data and jointly answer queries.

4 Protocol Description

This section first describes a strawman index design inspired from bloom filter (§4.1) and then introduces HoTEE's new MFT index based on the strawman index (§4.2). Then we present *SDA*, a secure data aggregation protocol that locates query-dependent parties using the MFT index and securely transforms multi-party data retrieved from hybrid encrypted databases into querier-decryptable data (§4.3).

4.1 A Strawman Search Index

Preliminaries. Bloom filter [12] is a space-efficient *monolithic* data structure that provides membership information: all elements are added to a single set and it checks whether an element is a member of a set. Also, a bloom filter has an intrinsic *confidential* characteristic: the elements themselves are not added to a set, instead the hash is added to the set. Formally, a bloom filter B is an array of m bits and parameterized by k independently uniform hash functions $\{H_k\}$, where $\{H_k\}$ map an element to k positions in B uniformly, and all elements are mapped to a single array monolithically.

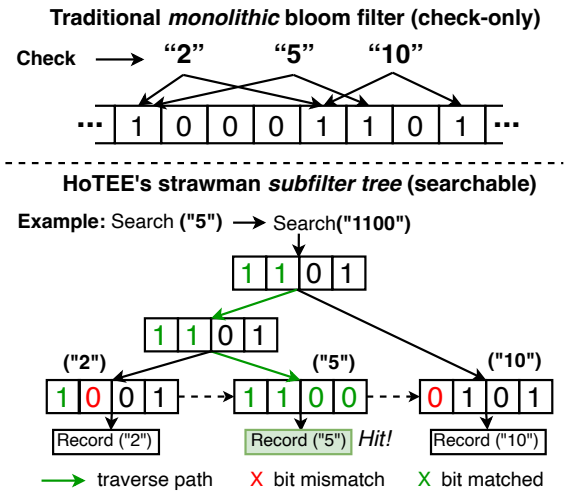


Figure 3: An initial search index built on top of subfilters.

A bloom filter supports confidential membership check. Specifically, to add an element x to B before checking, one uses the hash functions in $\{H_k\}$ to hash x and sets all bits at the position of hash values to `1`. Then, to check whether x is in the set, one also hashes x using the k hash functions. If any bit at the position of hash values is 0, x is definitely not in this set (none false negatives); otherwise, y is in the set (with small configurable false postives).

Overall, traditional monolithic bloom filter supports membership check using bit representation and inherits the confidentiality guarantee as long as we make standard assumptions on the one-wayness of cryptographic hash functions [22],

Subfilter-based search index. The traditional bloom filter supports only equality check by telling whether an element is in a set, however, it cannot serve as an index to locate the address of data in storage because of the monolithic design.

To this end, in our initial design, we divide a traditional monolithic bloom filter into *subfilters* where each subfilter stores the hash (i.e., bits) for only one element. As shown in Figure 3, subfilters have a constant size and are stacked into a merkle-tree style data structure for confidential data search. Specifically, each leaf node in the subfilter tree stores one subfilter of an index key (e.g., `"1001"` for index key *2*) and sorted in plaintext key order; for each inner node (all nodes except the last layer), its value is the element-wise *OR* gate computation result of its child nodes' subfilters: suppose subfilters have a constant size m, for an inner node *inner* with K child nodes cn_K, the inner node's subfilter is computed as:

$$inner[i] = cn_1[i] \; or \; ... \; cn_t[i] \; or \; ... \; cn_K[i] \; (\forall i \in [0, m]) \quad (1)$$

Same as B$^+$-tree, a subfilter tree's leaf nodes store values (e.g., record pointers) and inner nodes are only for indexing.

The rationale behind subfilter tree construction is that, if a `"1"` bit is present in a child node's subfilter, after *OR* com-

putation with other child nodes' subfilters, the parent (inner) node's subfilter must contain that "1" bit (e.g., if $cn_t[0]$ ="1", it holds $inner[0]$ ="1"). This implies HoTEE's new insight of using *existence* information for searching: if one element exists in a tree node (i.e., hashes of the element are set to "1" in the node's subfilter), the element must exist in the parent node's subfilter (i.e., all hashes of the element are "1"). This enables HoTEE to traverse a subfilter tree in a top-down manner by verifying the existence of query parameters in subfilter nodes with confidentiality, rather than compare the plaintext values as in traditional B$^+$-tree.

Based on the subfilter tree, to conduct a equality search, instead of searching with plaintext query parameters (for short, parameter), the client generates a confidential subfilter of the parameter using Equation 1, and then the database traverses the subfilter tree to locate the corresponding record of the parameter (Figure 3). Specifically, for each inner node in the subfilter tree, it compares with the parameter to verify whether every "1" bit of the parameter exists in the current node's subfilter; if so, the parameter is passed to the child nodes which conduct the same comparison till the leaf node. When reaching the leaf nodes, different from the verification in inner nodes, a leaf node must verify that every single bit in its subfilter matches the parameter because one subfilter comprises one element (i.e,. index key) exclusively.

The equality search on subfilter tree can be easily extended to range search. Given two range parameters (i.e., a left and right boundary), they are first transformed to two subfilters using Equation 1, then the tree searches two subfilters same as the equality search and locates two leaf nodes (if matched). Last, since leaf nodes are sorted during index construction, all leaf nodes between the two searched leaf nodes are returned as searched results.

4.2 HoTEE's MFT Search Index

Generality issue of subfilter-based search index. Although the subfilter tree (§4.1) can support equality and range queries with logarithmic-complexity through tree traversal, however, since index traversal uses existence verification rather than traditional numerical comparison, and there are no *orders* among subfilters, two fundamental limitations occur.

First, such an index cannot support general range queries. This is because, different from equality queries that directly search on query parameters, range queries can have parameters that are non-existent in databases. For example, suppose a database table has an `age` attribute with three values: 2, 5, 10, a range query could be `select [...] where 3 < age < 7`, in which 3 and 7 are non-existent. Trational B$^+$-tree can handle such a case because index nodes and query parameters are plaintexts, hence the position of non-existent keys can be trivially located by finding two adjacent values. However, since subfilter uses existence check and there are no numerical orders between subfilters, a subfilter tree cannot locate positions of non-existent keys through subfilter verification.

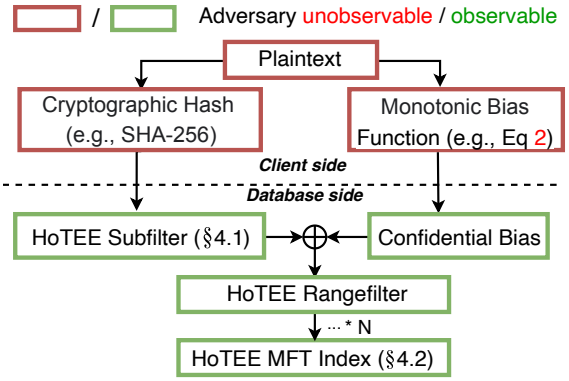


Figure 4: The routine to construct HoTEE's rangefilter primitive.

Second, owing to the same reason, a subfilter tree does not support write operations (e.g., insert), because the tree cannot locate the write position among sorted leaf nodes.

Design rationale. In order to support non-existent key search (for range queries) and writes, our key idea is that, instead of supporting specific functionalities using special encryption schemes as in CryptDB [37], HElibDB [4], we instead embedding the computation functionalities into data structure, which is build on top of traditional encryption schemes. Specifically, we identify that it is essential to have both *determinism* and *order* in one searchable data structure for supporting equality and range queries respectively, and the current subfilter tree design already constructs a searchable data structure with (1) determinism by creating subfilters with deterministic plaintext-ciphertext mappings, and (2) confidentiality that directly inherits from cryptographic hash functions in bloom filters. Hence, we should add the order property to subfilters, such that subfilters can preserve the same numerical orders as their plaintexts without sacrificing confidentiality.

HoTEE's rangefilter primitive. Based on the rationale, we propose *rangefilter*, a primitive for constructing confidential search indexes that support general queries by tackling the generality issues mentioned above. The routine to construct the rangefilter is shown in Figure 4. Specifically, to add the order property to HoTEE's subfilter, a client (data owner) generates a *confidential bias* of subfilter, and such a function should fulfil three requirements. First, the function should ensure monolithicity, which means that a larger plaintext should always have a larger bias. Secondly, to reduce the overhead at the client side, the generation process should be stateless, which means that the client should not store any states (e.g., previous biases) at runtime. Last, the function should provide randomness such that an adversary that observes a bias should not immediately deduce the plaintext.

Definition 1 (Confidential bias function). Given two plaintexts $p_1 > p_2$, if a function f satisfies: (1) $f(p_1) > f(p_2)$, (2) f is independent of any previous states, and (3) f varies within a certain range of values, then f is regarded as a qualified confidential bias function.

We formulate that a bias function f should have the form of $f(x)=g(x)+t(x,y,z)$, where $g(x)$ is monotonically increasing

Algorithm 1: MFT Index Preparation (Client side)

```
1  Input:
2      plaintext[]                          ▷ Plaintext keys for indexing
3      chashfunc[]    ▷ Cryptographic hash functions for filters
4      k, γ                                 ▷ Obfuscation parameters
5  Output:
6      rf[]      ▷ Rangefilters of plaintexts for encrypted search
7  Function Prepare() do
8      sortedPtx[] ← sort(plaintext)
       ▽ Generate subfilters
9      foreach plaintext p in sortedPtx do
10         subfilter[m] ← {0}      ▷ m is subfilter's bit length
11         foreach evaluator e in chashfunc do
12             subfilter.set(e(p), 1)
13         rf.insert(subfilter)
       ▽ Generate biases in plaintext order
14     range_min ← γ, range_max ← m + γ
15     start ← 0
16     foreach plaintext p in sortedPtx do
17         bias ← k * p + hash(p)              ▷ hash(p) ∈ [0,k)
18         p.rmin ← bias + range_min
19         p.rmax ← bias + range_max
20         rf[start++].rng ← {p.rmin, p.rmax}
21     return rf
```

on x, *t(x,y,z)* is a parametric function which varies within a certain range and has a fixed number of values, and does not overwhelm the monotonicity of $g(x)$. Any functions that fulfill the above formulation can be applied to generate biases for subfilters. In HoTEE, we use the bias function

$$f(x) = kx + hash(x),\ hash(x) \in [0,k) \tag{2}$$

in which x is the plaintext, k is a confidential digest, both are owned by the client. By applying bias b to a m length subfilter (i.e., m bits), a rangefilter has a range of $[b+\gamma, m+b+\gamma]$, in which γ is a random coin generated by the client to obfuscate adversaries.

Definition 2 (HoTEE rangefilter). A rangefilter R comprises a subfilter (§4.1) and a confidential bias (§4.2), providing data existence information for equality queries and order information for range queries, respectively. By integrating subfilter and bias into R, and using R as the index key for HE or TEE data, HoTEE supports general queries.

Note that, the entire process of generating rangefilters at client side is stateless because (1) the bias function (Equation 2) requires only a digest k and parameters of a hash function, and (2) subfilters requires only to store parameters of a fixed set of hash functions. Overall, to construct rangefilters, unlike previous work that requires clients to store stale states or even a full replica of outsourced data for encrypted query service [36], HoTEE supports lightweight clients without paying additional storage and computation overhead.

MFT confidential index. By using rangefilter as the smallest building block, we build Merkle Rangefilter Tree (MFT), an index that simutaneously supports equality and range searches on encrypted data with logarithmic-complexity. Below, we present details about MFT construction, MFT search operations, and robustness of MFT (e.g., query authentication).

Algorithm 2: MFT Index Construction (Client side)

```
1  Struct {
2      Rangefilter myFilter                 ▷ Input from Algo 1
3      Ctx value                        ▷ TEE or HE ciphertext
4      Node pn                                  ▷ Parent node
5  } MFTNode;
6  Function CreateLeaf() do
       ▽ Initialize bottom-layer MFTNodes
7      ...
8      return leaf[]
9  Function CreateInner(Node_i, Node_j) do
10     (inner.left, inner.right) ← (Node_i, Node_j)
11     (Node_i.pn, Node_j.pn) ← inner
       ▽ Union of child nodes' ranges
12     inner.rng ← Node_i.rng ∪ Node_j.rng
13     return inner
14 Function CreatMFTIndex() do
15     le[]← CreateLeaf(), in[]← ∅, ptr ← 0
       ▽ Pair-wise inner node construction
16     while ptr < le.length do
17         if ptr = le.length − 1 then
18             in.insert(CreateInner (le[ptr],le[ptr-1].pn))
19         else
20             in.insert(CreateInner (le[ptr],le[ptr+1]))
21         ptr ← ptr + 2
       ▽ Bottom-up MFT construction
22     while in.length > 2 do
23         n[]← ∅, ptr ← 0
24         while ptr < in.length do
25             if ptr = in.length − 1 then
26                 n.insert(CreateInner (in[ptr],in[ptr-1].pn))
27             else
28                 n.insert(CreateInner (in[ptr],in[ptr+1]))
29             ptr ← ptr + 2
30         in ← n
       ▽ Entry of MFT confidential index
31     return root ← CreateInner(in[0], in[1])
```

• **MFT construction. Algo** 1 prepares prerequisites at the client side, which takes client's plaintexts (i.e., index keys) as input, and outputs rangefilters of plaintexts for constructing MFT index by generating subfilters and biases respectively (as in Figure 4). Specifically, we first sort the plaintexts to ensure the encrypted data preserve the same numerical order as plaintexts (line 9). Then, we construct subfilters of sorted plaintexts using client-chosen (unknown to database) cryptographic hash functions (e.g., SHA-256) using Equation 1 (line 10∼14). After that, we generate biases for sorted plaintexts using Equation 2, add biases with obfuscation parameters and bit length for producing ranges (i.e., confidential order information) of subfilters (line 15∼20).

By using the rangefilters, a client constructs Merkle rangefilter tree (MFT) based on the dataset before uploading the index to the database, as shown in **Algo** 2. Concretely, the client creates MFT's leaf nodes (`MFTNode`) with rangefilters built on index keys and ciphertexts (either TEE or HE data), and creates MFT's inner nodes (line 9∼13).

Note that, similar to a traditional B^+-tree, there are two construction rules in MFT. Firstly, all leaf nodes are sorted in numerical orders as plaintexts and linked for fast data fetching.

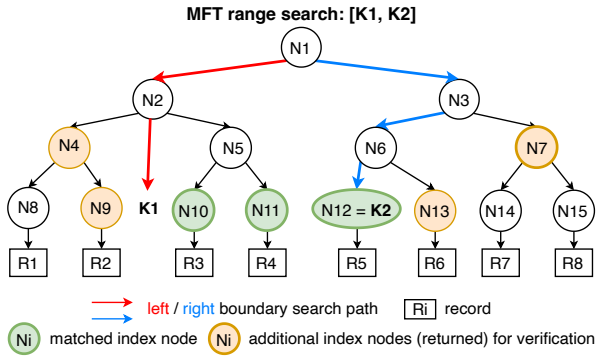


Figure 5: An MFT range search example.

Secondly, only the leaf nodes contain values (i.e., pointers of ciphertexts' to the storage) and other inner nodes are only for indexing (i.e., tree traversal). However, different from a traditional B$^+$-tree, an MFT rangefilter inner node has two components including a subfilter and a range. The subfilter is the *OR* gate computation result of child nodes; the range is the union of child nodes' ranges. We will introduce how the MFT uses rangefilter primitive to search later. Last, a MFT is constructed by stacking inner nodes iteratively, and finally a root entry of MFT is returned (line 14∼31).

Note that, **Algo** 2 constructs an index based on static data before sending queries, however, HoTEE also support inserting, updating, or deleting data dynamically at runtime by using search operations introduced below.

- **MFT confidential search.** In MFT, the high-level idea of encrypted search is to use the biased range for indexing (tree traversal) as it preserves the numerical orders of encrypted data as in plaintexts, and use the subfilter for point match as it contains the existence information of plaintexts (**Algo** 3). As the vast majority of database read/write queries are composed by equality and range operations [31], we introduce how HoTEE's MFT index handles equality and range searches below.

Specifically, for equality search, MFT compares the range of searched target and each tree node till the bottom layer, if the target's range is within the range of the tree node, it implies the the child nodes of that tree node may contain the searched target, because HoTEE uses the union computation for ranges when constructing parent nodes (**Algo** 2). Then, at the bottom layer, HoTEE compares to see if every bit in target's subfilter matches a leaf node's subfilter. Only when every single bit matches, the final result is searched (line 5∼14). This is because HoTEE's subfilter contains only one element and is exclusive to other elements, compared to a traditional monolithic bloom filter design (§4.1).

- **Using MFT for query authentication.**

Complexity analysis.

Security analysis.

Algorithm 3: MFT Confidential Search (Database side)

```
1  Input:
2    root                                  ▷ Entry of MFT from Algo 2
3    key (i.e., rangefilter)               ▷ Search target
4    opt (P | L | R)   ▷ Search option: Point search or range
       search with Left or Right boundary; default is P
5  Function EqualitySearch(root, key, opt=p) do
       ▽ Compare subfilters (§4.1) at leaf nodes
6      if isLeaf(root) then
           ▽ Every subfilter bit must match
7          if key.subfilter = root.subfilter then
8              return root
9          else
10             return null
       ▽ Use biased range to traverse MFT
11     if key.rng ∈ root.left.rng then
12         return EqualitySearch(root.left, key)
13     if key.rng ∈ root.right.rng then
14         return EqualitySearch(root.right, key)
       ▽ Case 1:left boundary is non-existent
15     if opt = l then
           ▽ Find smallest data larger than key
16         root ← root.right
17         while !isLeaf(root) do
18             root ← root.left
19         return root
       ▽ Case 2:right boundary is non-existent
20     if opt = r then
           ▽ Find largest data smaller than key
21         root ← root.left
22         while !isLeaf(root) do
23             root ← root.right
24         return root
25 Function RangeSearch(root, [key1, key2]) do
26     value[]← ∅
27     left ← EqualitySearch(root, key1, l)
28     right ← EqualitySearch(root, key2, r)
       ▽ Traverse linked leaves in order
29     while left.next != right do
30         value.insert(left.next)
31         left ← left.next
32     return value
```

4.3 Hybrid-trust Multi-party Joint Query

5 Implementation

6 Evaluation

7 Discussion

7.1 HoTEE's Generality

8 Conclusion

References

[1] Amazon AWS. 2022. State & local government. Retrieved 10/27/2022 from https://aws.amazon.com/stateandlocal/.

[2] Edgeless Systems GmbH. 2022. *EdgelessDB Official Website. Retrieved March 1, 2022*. https://www.edgeless.systems/products/edgelessdb.

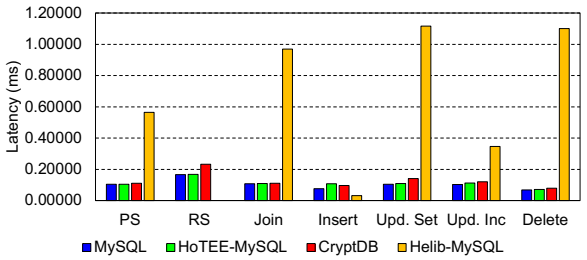


Figure 6

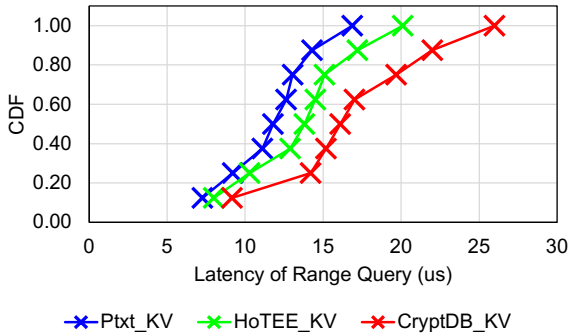


Figure 7

[3] Google Cloud. 2022. Harvard global health institute covid-19 forecast. Retrieved 10/27/2022 from https://cloud.google.com/blog/products/ai-machine-learning/google-cloud-covid-19-public-forecasts.

[4] HElib-KV Lookup. 2022. *HElib Country Lookup Example.* https://github.com/homenc/HElib/tree/master/examples/BGV_country_db_lookup.

[5] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. Order preserving encryption for numeric data. In *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*, pages 563–574, 2004.

[6] Panagiotis Antonopoulos, Arvind Arasu, Kunal D Singh, Ken Eguro, Nitish Gupta, Rajat Jain, Raghav Kaushik, Hanuma Kodavalla, Donald Kossmann, Nikolas Ogg, et al. Azure sql database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525, 2020.

[7] Arvind Arasu, Spyros Blanas, Ken Eguro, Manas Joglekar, Raghav Kaushik, Donald Kossmann, Ravi Ramamurthy, Prasang Upadhyaya, and Ramarathnam Venkatesan. Secure database-as-a-service with cipherbase. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1033–1036, 2013.

[8] Arvind Arasu, Spyros Blanas, Ken Eguro, Raghav Kaushik, Donald Kossmann, Ravishankar Ramamurthy, and Ramarathnam Venkatesan. Orthogonal security with cipherbase. In *CIDR*, 2013.

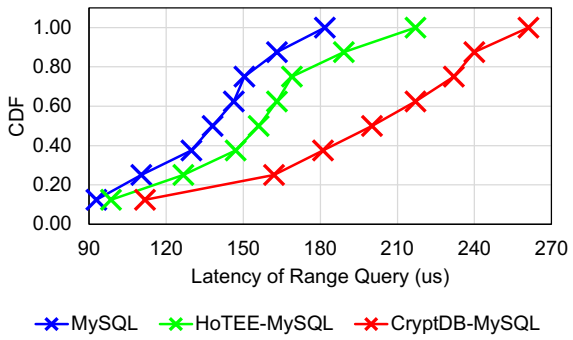


Figure 8

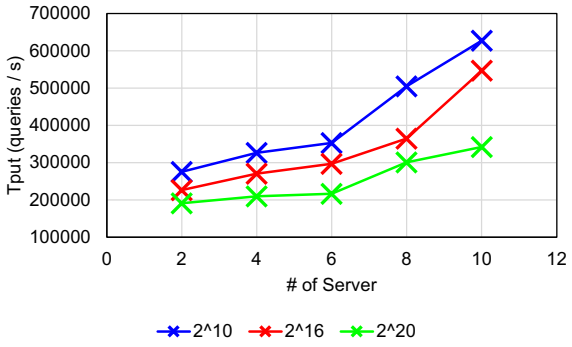


Figure 9

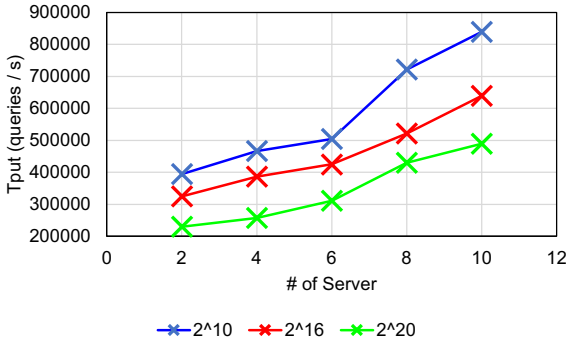


Figure 10

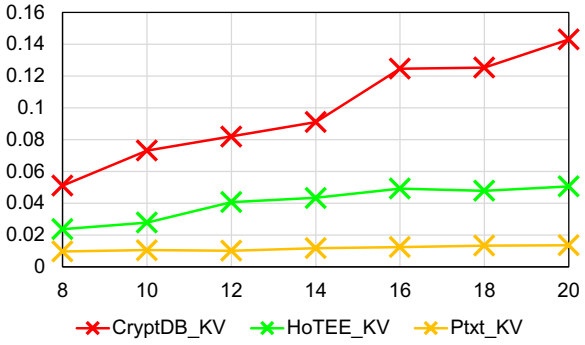


Figure 11

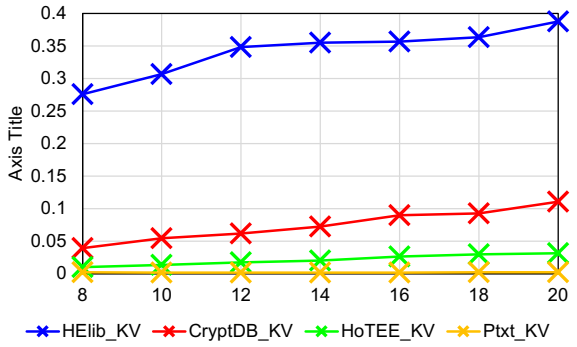


Figure 12

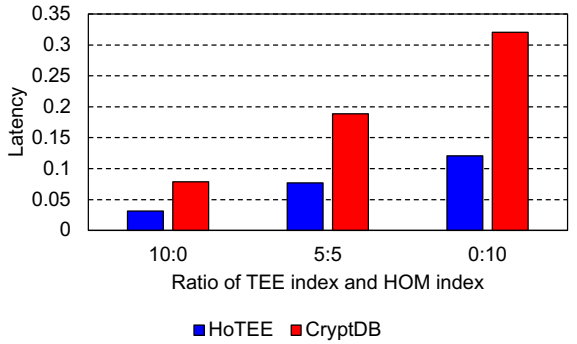


Figure 13

[9] David W Archer, Dan Bogdanov, Yehuda Lindell, Liina Kamm, Kurt Nielsen, Jakob Illeborg Pagter, Nigel P Smart, and Rebecca N Wright. From keys to databases—real-world applications of secure multi-party computation. *The Computer Journal*, 61(12):1749–1771, 2018.

[10] Mihir Bellare, Marc Fischlin, Adam O'Neill, and Thomas Ristenpart. Deterministic encryption: Definitional equivalences and constructions without random oracles. In *Annual International Cryptology Conference*, pages 360–378. Springer, 2008.

[11] Steven Michael Bellovin and William R Cheswick. Privacy-enhanced searches using encrypted bloom filters. 2007.

[12] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

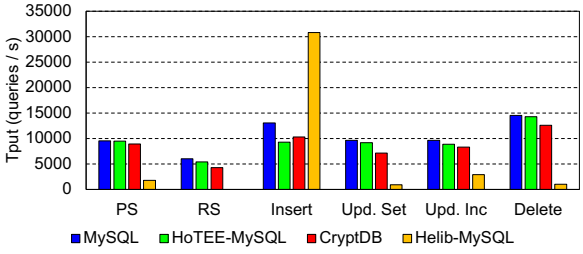


Figure 14

[13] Marcus Brandenburger, Christian Cachin, Matthias Lorenz, and Rüdiger Kapitza. Rollback and forking detection for trusted execution environments using lightweight collective memory. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 157–168. IEEE, 2017.

[14] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2):1–26, 2008.

[15] Rupali Chopade and Vinod Pachghare. Mongodb indexing for performance improvement. In *ICT Systems and Sustainability*, pages 529–539. Springer, 2020.

[16] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.

[17] Ronald Cramer, Ivan Bjerre Damgård, et al. *Secure multiparty computation*. Cambridge University Press, 2015.

[18] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. Snoopy: Surpassing the scalability bottleneck of oblivious storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 655–671, 2021.

[19] Emma Dauterman, Eric Feng, Ellen Luo, Raluca Ada Popa, and Ion Stoica. Dory: An encrypted search system with distributed trust. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1101–1119, 2020.

[20] Saba Eskandarian and Matei Zaharia. Oblidb: Oblivious query processing for secure databases. *arXiv preprint arXiv:1710.00458*, 2017.

[21] Craig Gentry, Shai Halevi, Chris Peikert, and Nigel P Smart. Ring switching in bgv-style homomorphic encryption. In *International Conference on Security and Cryptography for Networks*, pages 19–37. Springer, 2012.

[22] Vipul Goyal, Adam O'Neill, and Vanishree Rao. Correlated-input secure hash functions. In *Theory of Cryptography Conference*, pages 182–200. Springer, 2011.

[23] Paul Grubbs, Thomas Ristenpart, and Vitaly Shmatikov. Why your encrypted database is not secure. In *Proceedings of the 16th workshop on hot topics in operating systems*, pages 162–168, 2017.

[24] Shai Halevi and Victor Shoup. Algorithms in helib. In *Annual Cryptology Conference*, pages 554–571. Springer, 2014.

[25] Scott R. Intel. *SGX 2.0 (scalable SGX).* https://github.com/intel/linux-sgx/issues/899.

[26] Jianyu Jiang, Claudio Soriente, and Ghassan Karame. On the challenges of detecting side-channel attacks in sgx. In *25th International Symposium on Research in Attacks, Intrusions and Defenses*, pages 86–98, 2022.

[27] Emilia Käsper and Peter Schwabe. Faster and timing-attack resistant aes-gcm. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 1–17. Springer, 2009.

[28] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems*, 34:4961–4973, 2021.

[29] Scott T Leutenegger and Daniel Dias. A modeling study of the tpc-c benchmark. *ACM Sigmod Record*, 22(2):22–31, 1993.

[30] Dong Li, Xiaofeng Liao, Tao Xiang, Jiahui Wu, and Junqing Le. Privacy-preserving self-serviced medical diagnosis scheme based on secure multi-party computation. *Computers & Security*, 90:101701, 2020.

[31] Dongxi Liu and Shenlu Wang. Programmable order-preserving secure index for encrypted database query. In *2012 IEEE Fifth International Conference on Cloud Computing*, pages 502–509. IEEE, 2012.

[32] Yanbin Lu et al. Privacy-preserving logarithmic-time search on encrypted data in cloud. In *NDSS*, 2012.

[33] Murali Mani, Kinnari Shah, and Manikanta Gunda. Enabling secure database as a service using fully homomorphic encryption: Challenges and opportunities. *arXiv preprint arXiv:1302.2654*, 2013.

[34] AB MySQL. Mysql, 2001.

[35] Proteet Paul, Tushar Gupta, and Shamik Sural. Poster: Asql-attribute based access control extension for sql. In *Proceedings of the 27th ACM on Symposium on Access Control Models and Technologies*, pages 259–261, 2022.

[36] Rishabh Poddar, Tobias Boelter, and Raluca Ada Popa. Arx: A strongly encrypted database system. *IACR Cryptol. ePrint Arch.*, 2016:591, 2016.

[37] Raluca Ada Popa, Catherine MS Redfield, Nickolai Zeldovich, and Hari Balakrishnan. Cryptdb: protecting confidentiality with encrypted query processing. In *Proceedings of the twenty-third ACM symposium on operating systems principles*, pages 85–100, 2011.

[38] Behandelt PostgreSQL. Postgresql. *Web resource: http://www. PostgreSQL. org/about*, 1996.

[39] Christian Priebe, Kapil Vaswani, and Manuel Costa. Enclavedb: A secure database using sgx. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 264–278. IEEE, 2018.

[40] Mario Schkolnick. Secondary index optimization. In *Proceedings of the 1975 ACM SIGMOD international conference on Management of data*, pages 186–192, 1975.

[41] Meng Shen, Junxian Duan, Liehuang Zhu, Jie Zhang, Xiaojiang Du, and Mohsen Guizani. Blockchain-based incentives for secure and collaborative data sharing in multiple clouds. *IEEE Journal on Selected Areas in Communications*, 38(6):1229–1241, 2020.

[42] Erez Shmueli, Ronen Vaisenberg, Ehud Gudes, and Yuval Elovici. Implementing a database encryption solution, design and implementation issues. *Computers & security*, 44:33–50, 2014.

[43] Yuanyuan Sun, Sheng Wang, Huorong Li, and Feifei Li. Building enclave-native storage engines for practical encrypted databases. *Proceedings of the VLDB Endowment*, 14(6):1019–1032, 2021.

[44] Eugene Tsyrklevich and Bennet Yee. Dynamic detection and prevention of race conditions in file accesses. In *12th USENIX Security Symposium (USENIX Security 03)*, 2003.

[45] Stephen Lyle Tu, M Frans Kaashoek, Samuel R Madden, and Nickolai Zeldovich. Processing analytical queries over encrypted data. 2013.

[46] Harshavardhan Unnibhavi, David Cerdeira, Antonio Barbalace, Nuno Santos, and Pramod Bhatotia. Secure and policy-compliant query processing on heterogeneous computational storage architectures. In *ACM SIGMOD/PODS International Conference on Management of Data 2022*, 2022.

[47] Pengfei Wu, Jianting Ning, Jiamin Shen, Hongbing Wang, and Ee-Chien Chang. Hybrid trust multi-party computation with trusted execution environment.

[48] Yu Xia, Xiangyao Yu, Matthew Butrovich, Andrew Pavlo, and Srinivas Devadas. Litmus: Towards a practical database management system with verifiable acid properties and transaction correctness. In *Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA*, pages 12–17, 2022.

[49] Tao Yang, Jinming Wang, Weijie Hao, Qiang Yang, and Wenhai Wang. Hybrid cloud-edge collaborative data anomaly detection in industrial sensor networks. *arXiv preprint arXiv:2204.09942*, 2022.

[50] Statistics Canada. Zachary Zanussi. *Privacy Preserving Technologies Part Two: Introduction to Homomorphic Encryption.* https://www.statcan.gc.ca/en/data-science/network/homomorphic-encryption.